

STK1110 - Statistiske metoder og dataanalyse:

Obligatorisk øving 1 av 2

Vegard Nyquist Borgeraas

Høst 2026

Oppgave 1

(a)

Ved å sette de empiriske momentene lik de teoretiske, får vi følgende likningssystem:

$$M_1 := \frac{1}{n} \sum_{i=1}^n x_i \stackrel{!}{=} \frac{\alpha}{\gamma}$$

$$M_2 := \frac{1}{n} \sum_{i=1}^n x_i^2 \stackrel{!}{=} \text{Var}(X) + E[X]^2 = \frac{\alpha}{\gamma^2} + \left(\frac{\alpha}{\gamma}\right)^2$$

Løser vi dette systemet for de to ukjente parametrene, får vi momentestimatorene:

$$\hat{\gamma} = \frac{M_1}{M_2 - M_1^2}$$
$$\hat{\alpha} = \frac{M_1^2}{M_2 - M_1^2}$$

Resultatene samsvarer med boken, ettersom vi har at:

$$\hat{\gamma} = \frac{M_1}{M_2 - M_1^2} = \frac{1}{\hat{\beta}_{\text{bok}}}$$

Dette stemmer overens med at $\beta = 1/\gamma$.

R-kode for å beregne de empiriske momentene basert på forsikringsdataene:

```
data <- scan("forsikringskrav.txt")
```

```
M1 <- mean(data)
```

```
M2 <- mean(data^2)
```

```
alpha <- M1^2 / (M2 - M1^2)
```

```
gamma <- M1 / (M2 - M1^2)
```

Dette gir at $\hat{\gamma}_{\text{mom}} = 0.696649$ og $\hat{\alpha}_{\text{mom}} = 0.02886053$.

(b)

Log-likelihood-funksjonen, $l(\alpha, \gamma)$, finner vi ved å ta den naturlige logaritmen av likelihood-funksjonen. Ved å bruke vanlige logaritmereglene gjøres produktet om til en sum:

$$\begin{aligned} l(\alpha, \gamma) &= \sum_{i=1}^n \ln(f(x_i | \alpha, \gamma)) \\ &= \sum_{i=1}^n \ln \left(\frac{\gamma^\alpha}{\Gamma(\alpha)} x_i^{\alpha-1} e^{-\gamma x_i} \right) \\ &= \sum_{i=1}^n (\alpha \ln(\gamma) - \ln(\Gamma(\alpha)) + (\alpha - 1) \ln(x_i) - \gamma x_i) \\ &= n\alpha \ln(\gamma) - n \ln(\Gamma(\alpha)) + (\alpha - 1) \sum_{i=1}^n \ln(x_i) - \gamma \sum_{i=1}^n x_i \end{aligned}$$

R-kode for beregning av log-likelihood:

```
l <- function(x, alpha, gamma) {  
  n <- length(x)  
  n * alpha * log(gamma) - n * lgamma(alpha) +  
  (alpha - 1) * sum(log(x)) - gamma * sum(x)  
}
```

Dette gir at $l(\hat{\alpha}_{\text{mom}}, \hat{\gamma}_{\text{mom}}) = -27264.51$.

(c)

For å finne maksimum likelihood-estimatoren for γ tar vi den partiellderiverte av $l(\alpha, \gamma)$ med hensyn på γ , og setter uttrykket lik null:

$$\begin{aligned} \frac{\partial l}{\partial \gamma} &= \frac{\partial}{\partial \gamma} \left(n\alpha \ln(\gamma) - n \ln(\Gamma(\alpha)) + (\alpha - 1) \sum_{i=1}^n \ln(x_i) - \gamma \sum_{i=1}^n x_i \right) \\ &= \frac{n\alpha}{\gamma} - \sum_{i=1}^n x_i \end{aligned}$$

For å finne maksimumspunktet, setter vi den deriverte lik null:

$$\frac{n\alpha}{\gamma} - \sum_{i=1}^n x_i = 0$$

Løser vi denne ligningen for γ , får vi maksimum likelihood-estimatoren:

$$\hat{\gamma}_{\text{ML}} = \frac{n\alpha}{\sum_{i=1}^n x_i} = \frac{\alpha}{\frac{1}{n} \sum_{i=1}^n x_i} = \frac{\alpha}{\bar{x}}$$

Fra oppgave (a) har vi at:

$$\bar{x} = \frac{\alpha}{\gamma} \implies \gamma = \frac{\alpha}{\bar{x}}$$

Dermed samsvarer de to estimatorene.

(d)

R-kode for numerisk optimering:

```
negloglikgamma <- function(logalpha,x=x) {
  n <- length(x)
  alpha <- exp(logalpha)
  gamma <- alpha/mean(x)
  logL <- n*alpha*log(gamma)-n*lgamma(alpha)+
    (alpha-1)*sum(log(x))-gamma*sum(x)
  -logL
}

fit.ml <- optim(log(alpha), negloglikgamma, x=x, method="BFGS")

alpha_ml <- exp(fit.ml$par)
gamma_ml <- alpha_ml / mean(x)

logL_ml <- -fit.ml$value
```

Dette gir at $\hat{\gamma}_{ML} = 0.05751589$ og $\hat{\alpha}_{ML} = 1.388346$, og dermed at $l(\hat{\alpha}_{ML}, \hat{\gamma}_{ML}) = -26488.28$. Log-likelihood-verdien for ML-estimatene er helt rimelig fordi den er høyere enn verdien fra momentestimatene i (b), slik man alltid forventer når man maksimerer funksjonen.

(e)

R-kode for bootstrap-simulering:

```
B <- 1000 # Antall bootstrap-simuleringer
n <- length(x)

alpha_boot <- numeric(B)
gamma_boot <- numeric(B)

for (i in 1:B) {
  x_b <- sample(x, size = n, replace = TRUE)

  fit_b <- optim(log(alpha), negloglikgamma, x = x_b, method = "BFGS")

  alpha_boot[i] <- exp(fit_b$par)
  gamma_boot[i] <- alpha_boot[i] / mean(x_b)
}
```

Da får vi følgende anslag for standardfeilen (SE) til maksimum likelihood-estimatene:

$$SE(\hat{\alpha}_{ML}) \approx 0.0297$$

$$SE(\hat{\gamma}_{ML}) \approx 0.0019$$

Og vi får følgende 95 % konfidensintervaller:

$$95 \% \text{ KI for } \alpha : [1.335, 1.450]$$

$$95 \% \text{ KI for } \gamma : [0.0542, 0.0614]$$

(f)

R-kode for konfidensintervaller for μ :

```
mu_boot <- alpha_boot / gamma_boot

ci_mu_95 <- quantile(mu_boot, c(0.025, 0.975))
ci_mu_99 <- quantile(mu_boot, c(0.005, 0.995))
```

Da får vi følgende konfidensintervaller for forventningsverdien μ :

```
95 % KI for  $\mu$  : [23.43, 24.81]
99 % KI for  $\mu$  : [23.29, 24.96]
```

Vi tester hypotesene $H_0 : \mu = 25$ mot $H_a : \mu \neq 25$.

Siden $\mu = 25$ ligger utenfor det brede 99 % konfidensintervallet, forkaster vi nullhypotesen på signifikansnivå $\alpha = 0.01$. Ettersom 99 %-intervallet inneholder 95 %-intervallet, forkastes nullhypotesen i tillegg på signifikansnivå $\alpha = 0.05$.

Da får vi følgende konklusjon for P-verdien:

P-verdi < 0.01

Oppgave 2

a)

Siden den sanne variansen σ^2 er ukjent og vi antar at målingene er uavhengige og normalfordelte, benytter vi en t-fordeling med $n - 1 = 14$ frihetsgrader.

R-kode for utregning av konfidensintervall:

```
x <- c(525, 587, ..., 562)
n <- length(x)

x_bar <- mean(x)
s <- sd(x)
t_val <- qt(0.975, df = n - 1)

nedre <- x_bar - t_val * s / sqrt(n)
ovre <- x_bar + t_val * s / sqrt(n)
```

Datasettet gir følgende konfidensintervall for forventet antocyaninnhold, μ :

$$\begin{aligned} 95 \% \text{ KI for } \mu &= \bar{x} \pm t_{0.025,14} \frac{s}{\sqrt{n}} \\ &= [543.85, 575.48] \end{aligned}$$

b)

R-kode for simulering av dekningsgrad:

```
B <- 10000
n <- 15
mu_sann <- 558
sigma_sann <- 30

sim_data <- matrix(
  rnorm(B * n, mean = mu_sann, sd = sigma_sann),
  nrow = B,
  ncol = n
)

x_bar <- rowMeans(sim_data)
s <- apply(sim_data, 1, sd)
t_val <- qt(0.975, df = n - 1)
feilmargin <- t_val * s / sqrt(n)

nedre_grense <- x_bar - feilmargin
ovre_grense <- x_bar + feilmargin

dekker_mu <- (nedre_grense <= mu_sann) & (ovre_grense >= mu_sann)

andel <- mean(dekker_mu)
```

Andelen ble 0.9517, noe som er en god tilnærming til 95 %. Dette illustrerer definisjonen av et konfidensintervall: at omtrent 95 % av alle slike konstruerte intervaller faktisk vil inneholde den sanne forventningsverdien μ .

c)

R-kode for tilnærmet konfidensintervall:

```
feilmargin_stor <- 1.96 * s / sqrt(n)

nedre_grense_stor <- x_bar - feilmargin_stor
ovre_grense_stor <- x_bar + feilmargin_stor

dekker_mu_stor <- (nedre_grense_stor <= mu_sann) & (ovre_grense_stor >= mu_sann)

andel_stor <- mean(dekker_mu_stor)
```

Når vi bruker det tilnærmede intervallet, viser simuleringen at andelen synker til 0.9325. Dette stemmer med at denne tilnærmingen antar at vi har mer informasjon om standardavviket enn vi faktisk har, og dermed lager et for smalt intervall.

d)

For å konstruere et konfidensintervall for standardavviket σ i en normalfordeling, bruker vi kji-kvadrat-fordelingen. Vi vet at størrelsen $\frac{(n-1)S^2}{\sigma^2}$ er χ^2 -fordelt med $n - 1$ frihetsgrader. Ved å snu på dette uttrykket, får vi følgende formel for et 95 % konfidensintervall for σ :

$$\left[\sqrt{\frac{(n-1)s^2}{\chi_{0.975,14}^2}}, \sqrt{\frac{(n-1)s^2}{\chi_{0.025,14}^2}} \right]$$

```
chi_nedre <- qchisq(0.025, df = n - 1)
chi_ovre  <- qchisq(0.975, df = n - 1)

nedre_grense_sigma <- sqrt((n - 1) * s^2 / chi_ovre)
ovre_grense_sigma  <- sqrt((n - 1) * s^2 / chi_nedre)

dekker_sigma <- (nedre_grense_sigma <= sigma_sann) & (ovre_grense_sigma >= sigma_sann)
andel_sigma  <- mean(dekker_sigma)
```

Når vi kjører denne simuleringen finner vi at andelen er 0.9489, som er tilnærmet 95 %, akkurat som forventet.

e)

R-kode for simulering med t-fordelte data:

```
sim_data <- mu_sann + sigma_sann * matrix(
  rt(B * n, df=7),
  nrow = B,
  ncol = n
)
```

Når vi kjører simuleringen er andelen 0.9511, som fremdeles ligger svært nær 95 %.

Dette resultatet viser at metoden for å lage konfidensintervall for forventningsverdien er svært robust mot avvik i antakelsen om normalfordeling.

f)

R-kode for konfidensintervall for standardavviket med t-fordelte data:

```
sigma_tilde <- sigma_sann * sqrt(7 / (7 - 2))  
dekker_sigma_tilde <- (nedre_grense_sigma <= sigma_tilde) & (ovre_grense_sigma >= sigma_tilde)  
andel_sigma_tilde <- mean(dekker_sigma_tilde)
```

Andelen ble 0.8823, som er betydelig lavere enn 95 %. Dette viser at metoden for å lage konfidensintervall for standardavviket er mye mer sensitiv for avvik fra antakelsen om normalfordeling.